

第一章：启动与初始化

一、核心概念

1.1 特权级 (Privilege Levels)

现代处理器将 CPU 执行模式分为多个特权级，限制不同层次软件对硬件的访问能力。这是操作系统安全隔离的硬件基础。

架构	特权级划分	说明
x86	Ring 0 ~ Ring 3	Ring 0 = 内核, Ring 3 = 用户
ARM	EL0 ~ EL3	EL0 = 用户, EL1 = OS, EL2 = Hypervisor, EL3 = Secure Monitor
RISC-V	M / S / U	M = 机器模式, S = 监督模式 (OS), U = 用户模式

关键原则：低特权级不能直接执行高特权级指令（如操作 CSR、关中断、修改页表基址），必须通过陷入（trap/ecall）请求高特权级代理完成。

1.2 固件 (Firmware)

固件是系统上电后第一个运行的软件，运行在最高特权级。职责包括：

- 硬件自检和初始化（内存控制器、时钟等）
- 建立基本的运行环境（设置中断委托、内存保护）
- 将控制权移交给操作系统内核

不同架构的固件体系：

架构	固件	内核与固件的接口
x86	BIOS / UEFI	INT 10h / UEFI Runtime Services
ARM	ARM Trusted Firmware	PSCI (Power State Coordination Interface)
RISC-V	OpenSBI / RustSBI	SBI (Supervisor Binary Interface)

1.3 引导流程 (Boot Sequence)

通用的引导流程可抽象为三个阶段：

上电 → 固件初始化 (最高特权级) → 加载内核到内存 → 跳转到内核入口 (降权) → 内核初始化 → 启动第一个用户进程

多核启动的核心问题：全局状态（内存分配器、页表、进程表）只能被初始化一次。因此需要一种“门控”机制：

- **BSP** (Bootstrap Processor)：负责全局初始化的主核
- **AP** (Application Processor)：等待 BSP 完成后再加入

1.4 链接脚本 (Linker Script)

链接脚本定义了程序在内存中的布局：代码段、数据段、BSS 段的起始地址和排列顺序。对内核来说，链接脚本决定了内核被加载到内存的哪个位置。

1.5 第一个用户进程

内核初始化完成后，需要创建并运行第一个用户态进程。这个进程是所有后续用户进程的祖先：

- Unix/Linux 中是 `init` (PID 1)
- 它由内核直接构造（不是通过 `fork`），是唯一一个“无中生有”的进程

二、基本推论

推论 1：初始化必须有严格的顺序依赖。 串口驱动必须在日志系统之前初始化（否则无法打印调试信息）；物理内存分配器必须在页表之前初始化（建立页表需要分配页）；页表必须在进程表之前初始化（进程需要映射内核栈）。这形成了一条不可打乱的依赖链。

推论 2：多核启动不能并行初始化全局状态。 如果两个核同时初始化物理内存分配器，会导致数据竞争。因此必须有一个核先完成全部全局初始化，其余核等待信号后只初始化自己的 per-core 状态。

推论 3：固件和内核之间需要明确的约定。 包括：内核的加载地址、入口函数的参数（核 ID、设备树地址等）、哪些中断/异常已经被委托给内核处理。这些约定构成了一种 ABI。

推论 4：BSS 段必须在使用前清零。 C 语言规定未初始化的全局变量为零。内核启动时 BSS 段的内容是未定义的，必须由启动代码显式清零，否则全局变量的初始值不可预测。

三、Linux / 通用实现

x86 + UEFI 的启动流程

```
上电 → UEFI (SEC → PEI → DXE → BDS)
      → 加载 bootloader (GRUB)
      → GRUB 加载 vmlinuz + initrd 到内存
      → 跳转到 Linux 内核入口 (startup_64)
      → 内核解压 → start_kernel()
      → 依次初始化各子系统
      → 创建 init 进程 (PID 1)
      → 运行 /sbin/init (systemd)
```

Linux 的 `start_kernel()` 初始化顺序（简化）：

1. `setup_arch()` — 架构相关初始化，解析设备树/ACPI
2. `mm_init()` — 内存管理（伙伴系统、slab）
3. `sched_init()` — 调度器
4. `init_IRQ()` — 中断控制器
5. `console_init()` — 控制台
6. `vfs_caches_init()` — VFS 缓存

7. `rest_init()` → 创建 `kernel_init` 内核线程 → `run_init_process()`

Linux 的多核启动：BSP 在 `start_kernel()` 中完成全部初始化后，通过 `smp_init()` 唤醒 AP。AP 的启动路径是 `start_secondary()`，只做 per-CPU 初始化（本地 APIC、调度器等），然后进入 idle 循环。

ARM 的多核唤醒方式：

- **spin-table：**AP 在一个物理地址上自旋等待，BSP 往该地址写入启动函数地址
- **PSCI：**通过 ATF 固件的 `CPU_ON` 调用唤醒 AP

第一个用户进程

Linux 的 `kernel_init` 线程依次尝试执行 `/sbin/init`、`/etc/init`、`/bin/init`、`/bin/sh`。现代 Linux 通常由 systemd 接管 PID 1 的职责。

四、RUOS 的实现

核心文件

- `src/boot/entry.S` — 裸机入口，设栈，跳 main
- `src/boot/main.c` — 内核 `main()`，顺序初始化各子系统
- `src/linker/kernel.ld` — 链接脚本，定义内核内存布局
- `src/boot/initcode.S` — 将 `user/initcode.bin` 嵌入内核镜像

4.1 从 OpenSBI 到 RUOS

RUOS 运行在 RISC-V S 态，由 OpenSBI 固件从 M 态移交控制权。

```

QEMU 上电
↓
OpenSBI (M-mode, 0x80000000)
├── 初始化 CLINT (定时器/IPI)
├── 设置 PMP (物理内存保护)
├── 委托异常/中断到 S 态 (medeleg, mideleg)
├── 设 mepc = 0x80200000
└── mret → S-mode
    ↓
RUOS _entry (S-mode, 0x80200000)
寄存器约定: a0 = hartid, a1 = FDT 地址
  
```

设计决策：不自己写 M 态代码，完全依赖 OpenSBI。

优点：大幅简化内核，避免处理 M 态的复杂硬件初始化（PMP、CLINT、中断委托）。

缺点：内核受限于 SBI 提供的能力，不能自由控制 M 态资源。但对教学 OS 来说这是合理的取舍。

4.2 多核门控

```
hart 0: 清 BSS → 设栈 → main() → 初始化所有子系统 → started=1
hart N: 设栈 → main() → spin 等 started → 初始化 per-hart 状态 → scheduler()
```

用一个 `volatile int started` 做屏障。与 Linux 的 `smp_init()` / ARM 的 spin-table 思路一致，但更简单——没有内存屏障指令（依赖 volatile 的语义），也没有 IPI 唤醒机制（纯自旋等待）。

对比 Linux：Linux 用 `cpu_online_mask` 位图 + `smp_wmb()` 内存屏障确保 AP 安全加入。RUOS 的 volatile 方案在 RISC-V 弱内存模型下理论上不够严谨，但在 QEMU 环境下实际可用。

4.3 初始化顺序

```
consoleinit();    // 1. 串口 → 才能 printf
printfinit();    // 2. printf 锁
kinit();          // 3. 物理内存分配器 (buddy+slab)
kvminit();        // 4. 内核页表 (需要 kalloc)
kvminithart();    // 5. 写 satp, 开启分页
procinit();       // 6. 进程表 (需要分页, 映射内核栈)
trapinit();       // 7. 中断向量
trapinithart();   // 8. 写 stvec
plicinit();       // 9. PLIC 全局初始化
plicinithart();   // 10. 当前 hart 的 PLIC
fileinit();       // 11. 文件表
iinit();          // 12. inode 缓存
binit();          // 13. buffer 缓存
virtio_disk_init();// 14. VirtIO 磁盘
userinit();       // 15. 创建第一个用户进程
```

依赖链：`console` → `kinit` → `kvminit` → `procinit` 是严格顺序。这与 Linux 的 `start_kernel()` 逻辑一致——都遵循“先有内存分配，再有虚拟内存，再有进程”的依赖链。

4.4 entry.S 的栈分配

```
la sp, stack0      # BSS 中预留的数组
li a2, 4096
addi a3, a0, 1      # a0 = hartid
mul a2, a2, a3      # (hartid+1) * 4096
add sp, sp, a2      # sp 指向当前 hart 栈顶
```

每个 hart 一页（4KB）早期内核栈，够用到 `procinit()` 分配正式内核栈。

对比 Linux：Linux 的早期栈也是静态分配在 BSS 中（`init_thread_union`），大小为 `THREAD_SIZE`（通常 8KB 或 16KB）。

4.5 第一个用户进程

`initcode.S` 用 `.incbin "user/initcode"` 把用户态二进制嵌入内核数据段。`userinit()` 复制到第一个进程的用户空间，设 `epc` 为入口地址。

对比 Linux：Linux 也有类似机制——早期 Linux 用 `initramfs` 嵌入内核镜像，解压后 `exec /init`。RUOS 的 `initcode.bin` 是纯二进制（`objcopy -O binary`），直接 `memcpy` 到用户页，比 Linux 的 ELF/cpio 解析简单得多。

4.6 链接脚本

```
ENTRY(_entry)
. = 0x80200000;      /* OpenSBI 约定的 S 态内核入口 */
.text : { ... }
.rodata : { ... }
.data : { ... }
.bss : { PROVIDE(bss_start = .); ... PROVIDE(bss_end = .); }
```

`0x80200000` 是 OpenSBI 的约定。`bss_start/bss_end` 供 `entry.S` 清零 BSS。`PROVIDE(end = .)` 标记内核结束地址，`kinit()` 从这里开始管理空闲物理内存。

五、设计取舍总结

决策	RUOS 选择	Linux / 通用做法	权衡
M 态代码	不写，依赖 OpenSBI	Linux 也依赖固件（UEFI/ATF）	教学 OS 的合理简化
多核同步	volatile 变量自旋	内存屏障 + IPI 唤醒	简单但理论上不够严谨
初始化顺序	严格串行	部分可并行（但没必要）	可靠性优先
第一个进程	纯二进制嵌入	ELF/initramfs	简单直接
内核加载地址	硬编码 0x80200000	由 bootloader 动态决定	固定环境下够用

六、八股 / 面试高频问题

Q: 操作系统的启动流程？ A: 上电 → 固件初始化（BIOS/UEFI/OpenSBI）→ 加载 bootloader → 加载内核到内存 → 内核初始化（内存管理→页表→进程→中断→文件系统）→ 创建第一个用户进程（init）。

Q: 多核 CPU 如何启动？BSP 和 AP 是什么？ A: BSP（Bootstrap Processor）是主核，负责全局初始化。AP（Application Processor）是从核，等待 BSP 完成后通过 IPI/spin-table/PSCI 唤醒，只做 per-CPU 初始化。

Q: 内核的虚拟地址和物理地址是什么关系？ A: 内核通常做恒等映射（`virtual = physical`）或固定偏移映射。这样内核可以通过简单计算在虚拟地址和物理地址之间转换（`pa = va - KERNBASE`）。

Q: 什么是链接脚本？为什么内核需要它？ A: 链接脚本定义程序在内存中的布局（`.text/.data/.bss` 的起始地址和排列）。内核的加载地址必须与固件约定一致（如 RISC-V 的 `0x80200000`），否则无法正确启动。

Q: BSS 段是什么？为什么要清零？ A: BSS 段存放未初始化的全局/静态变量。C 标准规定它们初始值为零，但 ELF 文件不存储它们的内容（节省空间），启动代码必须显式清零。

七、项目贡献亮点

- 理解 OpenSBI → S 态内核的完整交接流程（中断委托、PMP 配置、mepc 设置）
 - 实现多核门控启动（volatile started 信号），理解 BSP/AP 同步机制
 - 内核初始化的严格依赖链设计（console→kalloc→kvminit→procinit）
-

八、常见问题

Q: 如果 hart 0 在初始化中 panic 了，其他 hart 怎么办？ A: 它们会永远自旋在 `while(started == 0)` 上。RUOS 没有跨核 panic 广播机制。Linux 通过 NMI（Non-Maskable Interrupt）广播 panic 到所有核。

Q: 为什么 `start.c` 还保留着 M 态代码？ A: 历史遗留。某些 QEMU 配置（`-bios none`）可能走 M 态路径。保留是为了兼容性，但当前主流路径是 OpenSBI。